die Mobiliar

Container als Immutable Infrastructure

John M. Hutchison

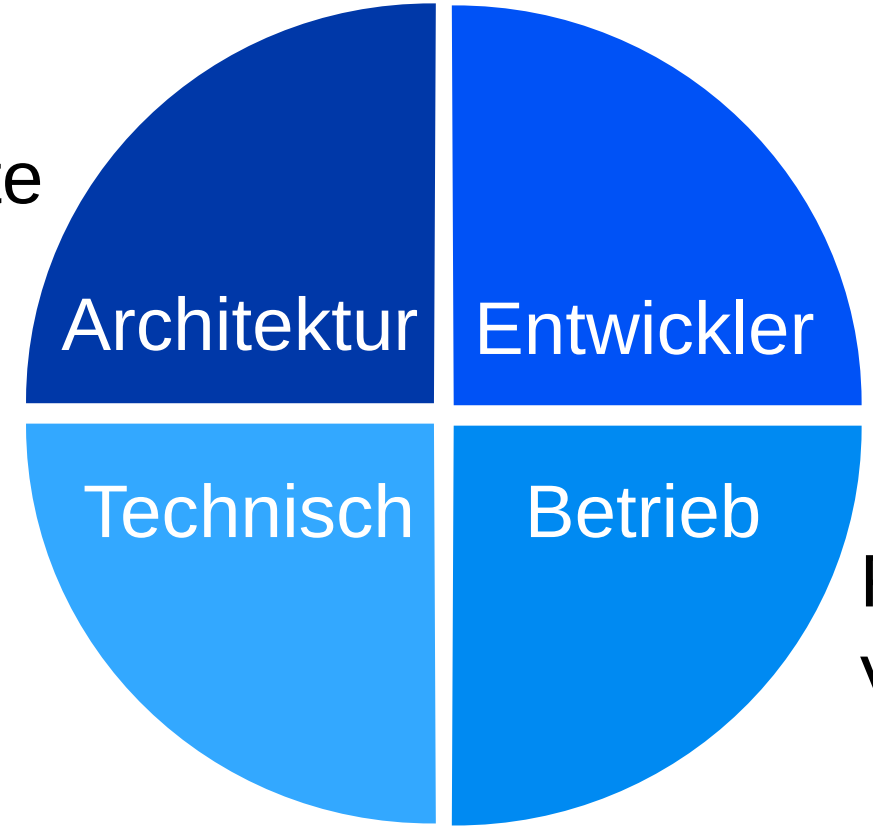
Container als Immutable Infrastructure

1. **Context**
2. **Anwendungsbereiche**
3. **Demo**
4. **Erkenntnisse**

Container – Verschiedene Sichten

Service, Komponente

Build Artefakt,
Deliverable



Linux Prozess mit
Einschränkungen

Ressourcen Template,
VM Provisionierung

Was bedeutet Immutable?

- Keine Anpassung an erzeugten Komponenten
- Komponenten werden einmalig erstellt
- Komponenten sind versioniert zentral verfügbar
- Services werden bei Änderung ganz neu verteilt
- Alte Service Instanzen werden abgebaut
- Nur die Konfiguration kann verändert werden

Auswirkungen von Immutable Infrastructure

- Reproduzierbarkeit
- Testbarkeit
- Transparenz
- Automatisierbarkeit

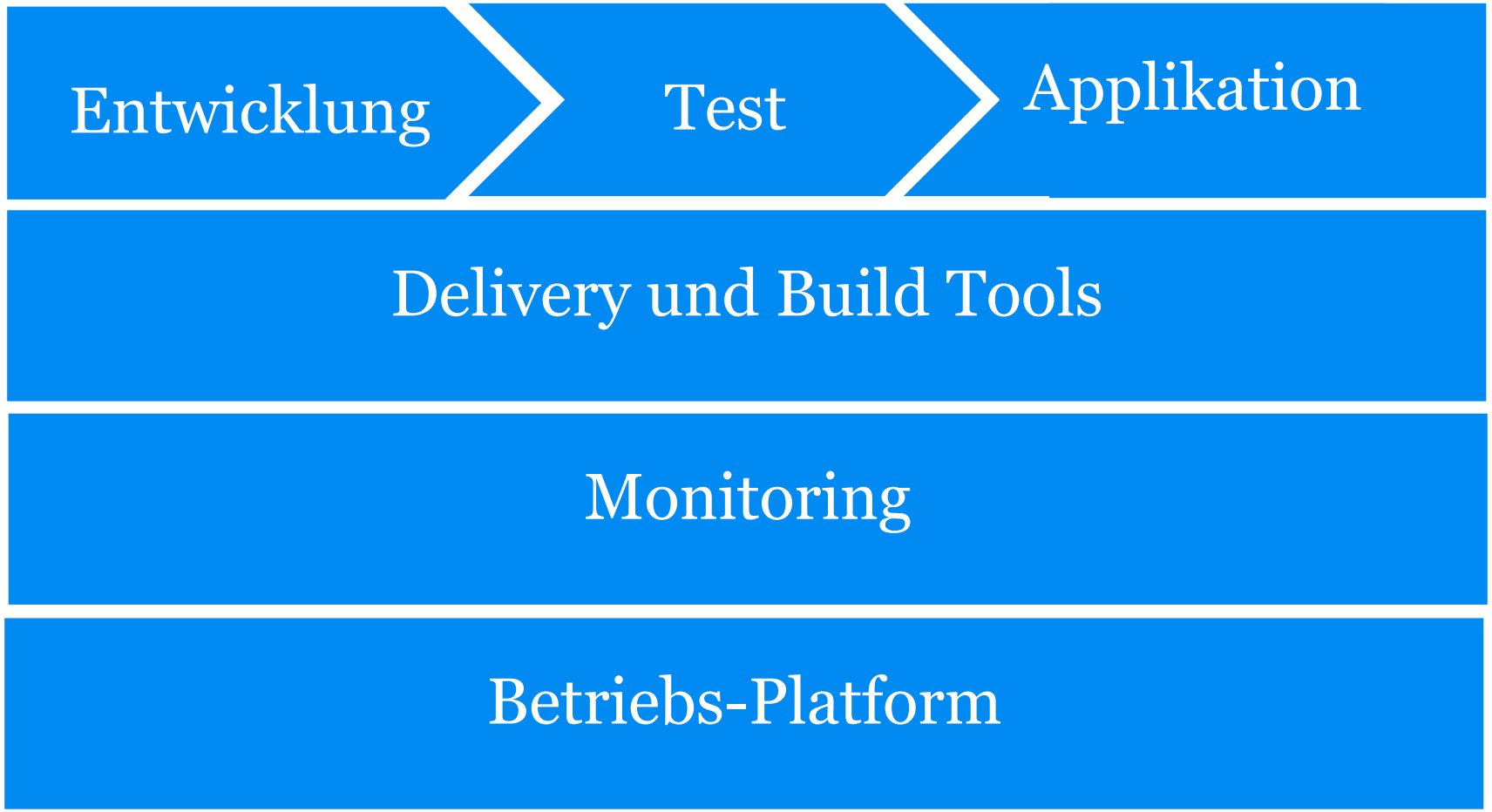
- Services müssen beliebig gestoppt und gestartet werden können.
- Betriebszustand darf nicht in der Komponente behalten werden => Stateless Services
- Time-to-Start ist wichtiger als Uptime

Container als Immutable Infrastructure

Container-Technologie vereinfacht alle wichtigen Aspekte von Immutable Infrastructure:

- Versionierung ist eingebaut
- Einmal erstelle Images können nicht mehr verändert werden
- Images sind leichtgewichtig und einfach zu starten
- Runtime-Abstraktion vereinfacht das Deployment
- Abbau von Container ist einfach und schnell

Infrastruktur



Fachapplikaiton

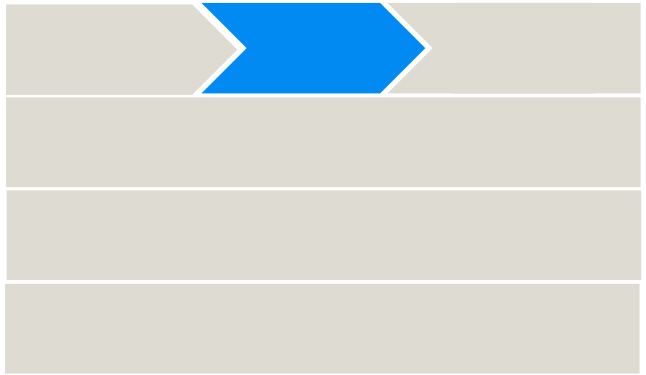
Anstatt Software Paket oder Artefakt wird ein Container Image ausgeliefert

- Paradigma bleibt gleich wie mit anderen Deliverables
- Infrastruktur-Bereitstellung kann beschleunigt werden
- Aufbau auf Basis-Images, um Patches und Updates an der Infrastruktur zu ermöglichen.

Testdaten

Datenbank mit vorbereiteten Daten in Images oder Volumes abgelegt

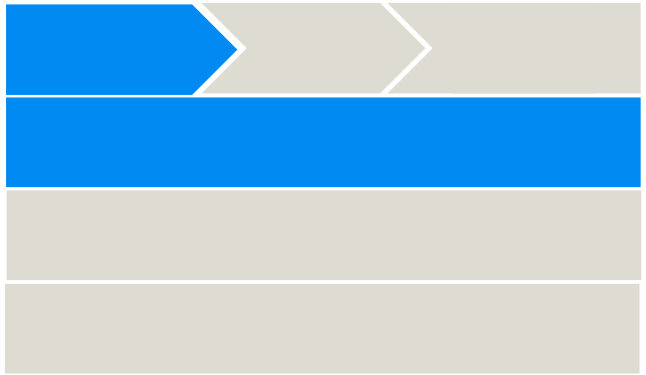
- Reproduzierbare Testdaten
- Schnelle Verfügbarkeit
- Testdaten-Verwaltung durch Versionierung vereinfacht



Build Tooling

Build-Tools werden in Images verpackt und zentral bereitgestellt

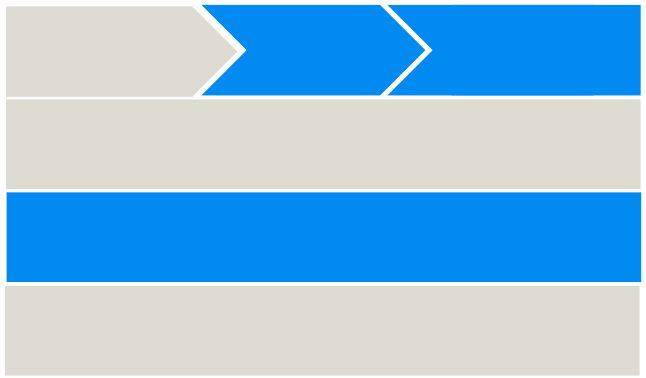
- Unabhängigkeit installierten verschiedener Versionen
- Unabhängigkeit von der Build-Infrastruktur
- Reproduzierbarkeit



Monitoring

Monitoring Tools werden in Images eingebaut um Daten zu sammeln

- In Basis-Images wo sie parallel laufen
- Als Companion Container
- Monitoring Services in Container betrieben
- Vereinfachte Updates von Standardtools
- Einfaches Ausprobieren neuer Tools



Delivery Pipeline

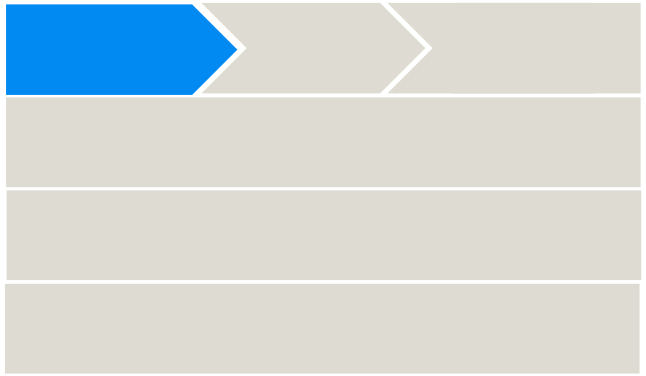
Delivery Pipeline Tools werden in Images verpackt, konfiguriert und betrieben

- Skalierung
- Testbarkeit
- Freiheit verschiedene Tools parallel zu betreiben
- Upgrades werden vereinfacht

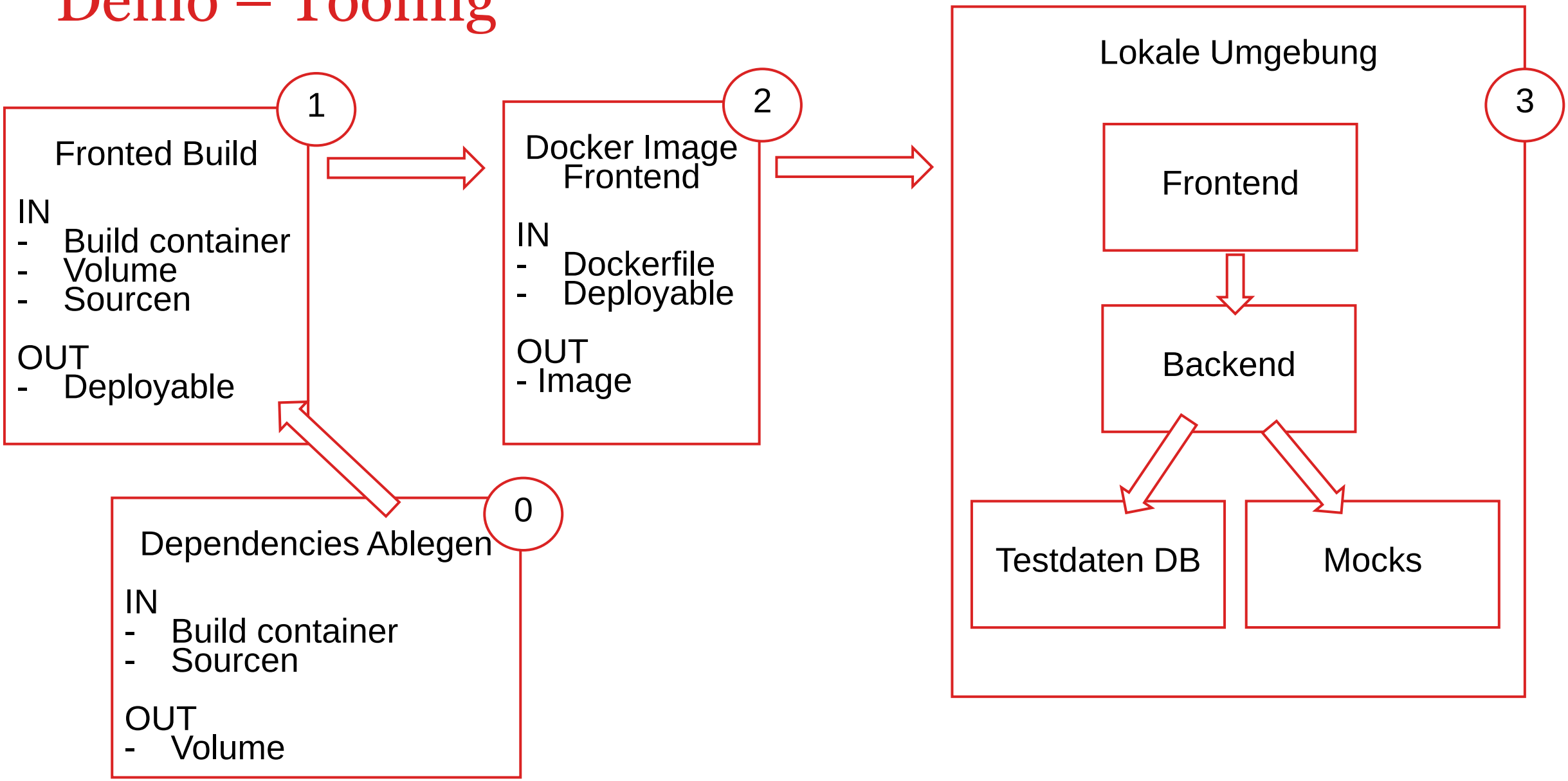
Entwicklungsumgebung

Entwicklungsumgebung kann in einem Image verpackt, für die Umgebung konfiguriert und abgelegt werden

- Beschleunigt Ramp-up neuer Entwickler und Rechner
- Firmen-Spezifische Setups zentral verwaltbar
- Projektunabhängigkeit
- Tool Unabhängigkeit
- Installations-Unabhängigkeit



Demo – Tooling



Erfahrungen

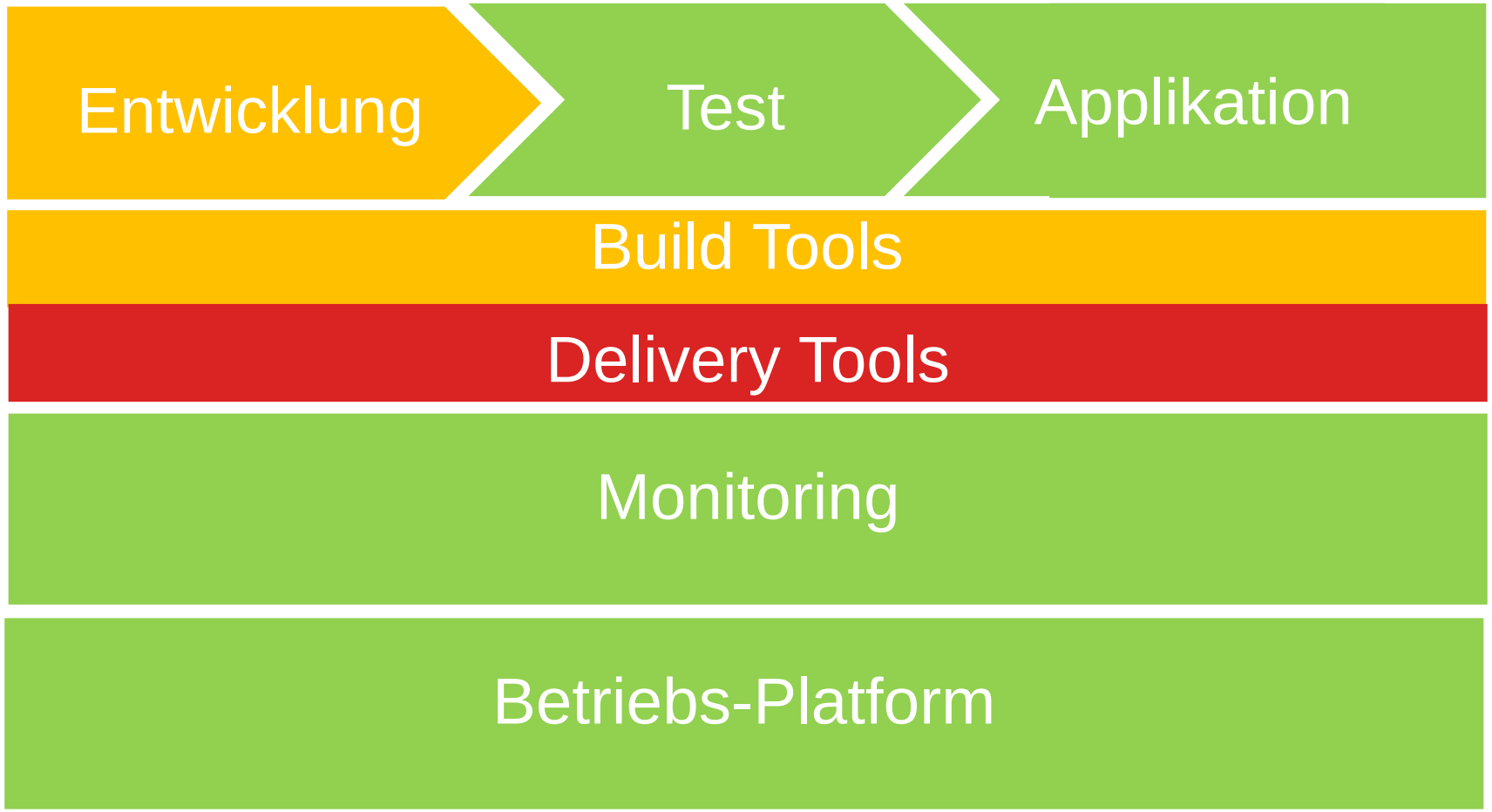
- Containerisierung bringt aus der Entwicklung bekannte Vorgehen zu Infrastruktur und Tools:
  Build, Test, Deploy sieht auf allen Stufen ähnlich aus

- Einführung von Container ist nicht gratis. Architektur, Vorgehensweisen und Mitarbeiter Skills müssen adressiert werden.

- Komplexität wird erhöht, es braucht gute Monitoring Tools, um diese in den Griff zu bekommen. Diese Arbeit erhöht aber zugleich die Maturität der gesamten Lösung.

- Druck wird erhöht auf andere IT-Einheiten, um mit der Flexibilität und Geschwindigkeit mitzuhalten.

Wo wir heute stehen

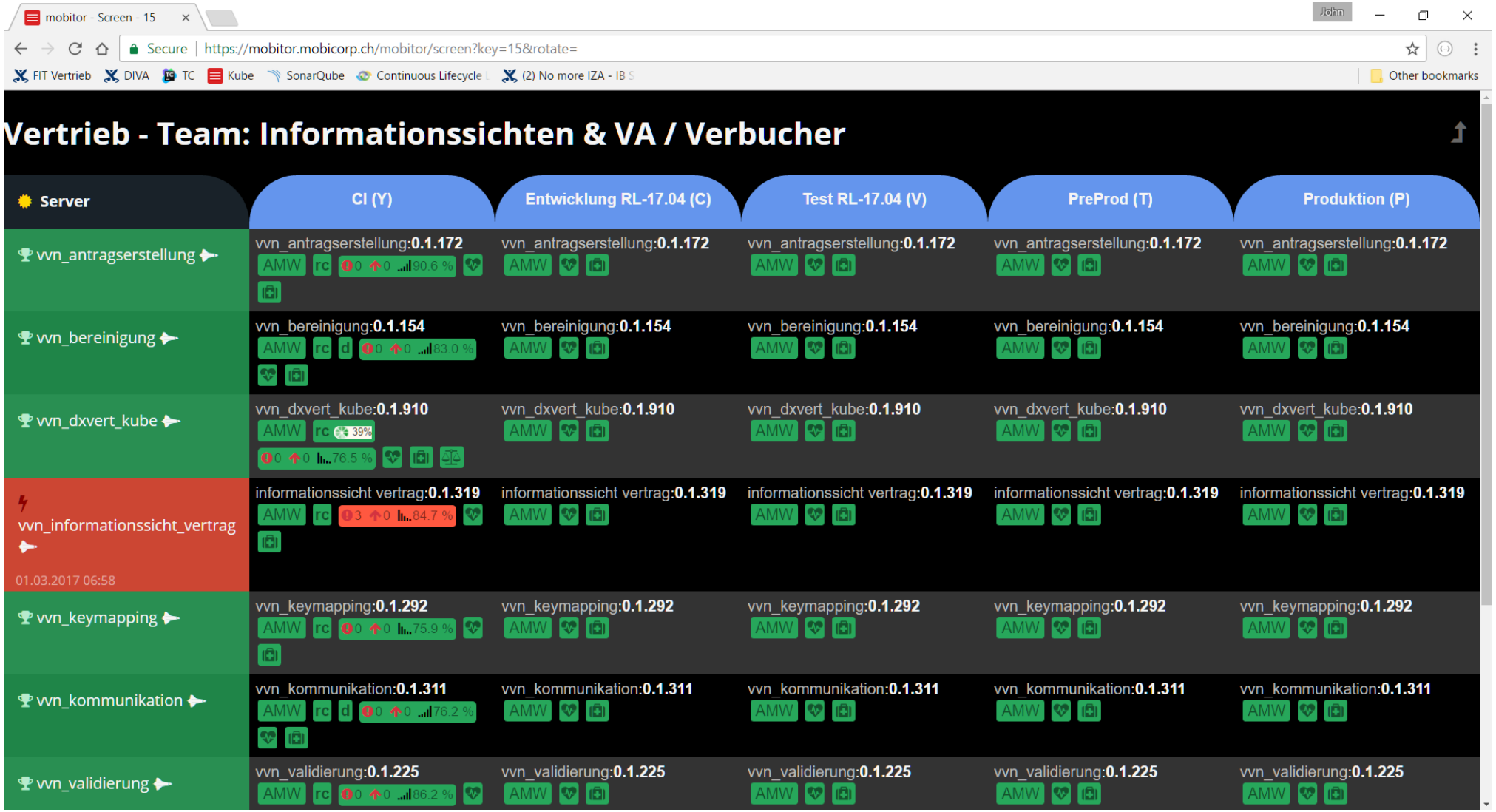


- Contanerisiert
- Teilweise containerisiert
- Nicht containerisiert

Wo wir heute stehen

- Neues Projekt bis und mit Deployment von 2-3 Wochen auf 1 Tag reduziert.
- Deployment von Commit bis Produktion in < 15min möglich.
- Datenbank Integration Tests reduziert von 30min auf 10sek.
- Datenbank as a Service ermöglicht neue Datenbank in < 5min, auch in Produktion

Monitoring Pipeline Übersicht



Zusammenfassung

- Container sind eine Technologie, die perfekt zu Immutable Infrastructure passt.
- Vorgehen erhöht; Testbarkeit, Transparenz, Deployment-Geschwindigkeit und Freiheit.
- Schaut Container immer von verschiedenen Seiten an, um neue Verwendungsmöglichkeiten zu entdecken.

Fragen

Informationssichten & VA / Verbucher
CI (Y)
Entwicklung RL-17.04 (C)
vvn_bereinigung
vvn_dxvert_kube
vvn_informationssicht_vertrag
vvn_keymapping
vvn_kommunikation
vvn_validierung